

Solution Design Document: Multi-Agent Competitor Analysis System

Date: February 2025

Author: Alif Elham Khan

1. Introduction

The Multi-Agent Competitor Analysis System is designed to automatically generate a comprehensive SWOT analysis for a given company using minimal user input (e.g., a company name such as "Adobe" or "Optimizely"). The system collects and enriches data from multiple trusted sources (Crunchbase, LinkedIn, Google Custom Search, G2) and leverages an advanced Large Language Model (LLM) to synthesize the information into a detailed SWOT analysis report.

This document outlines the architecture, design decisions, and alternative approaches with their respective trade-offs.

2. System Overview

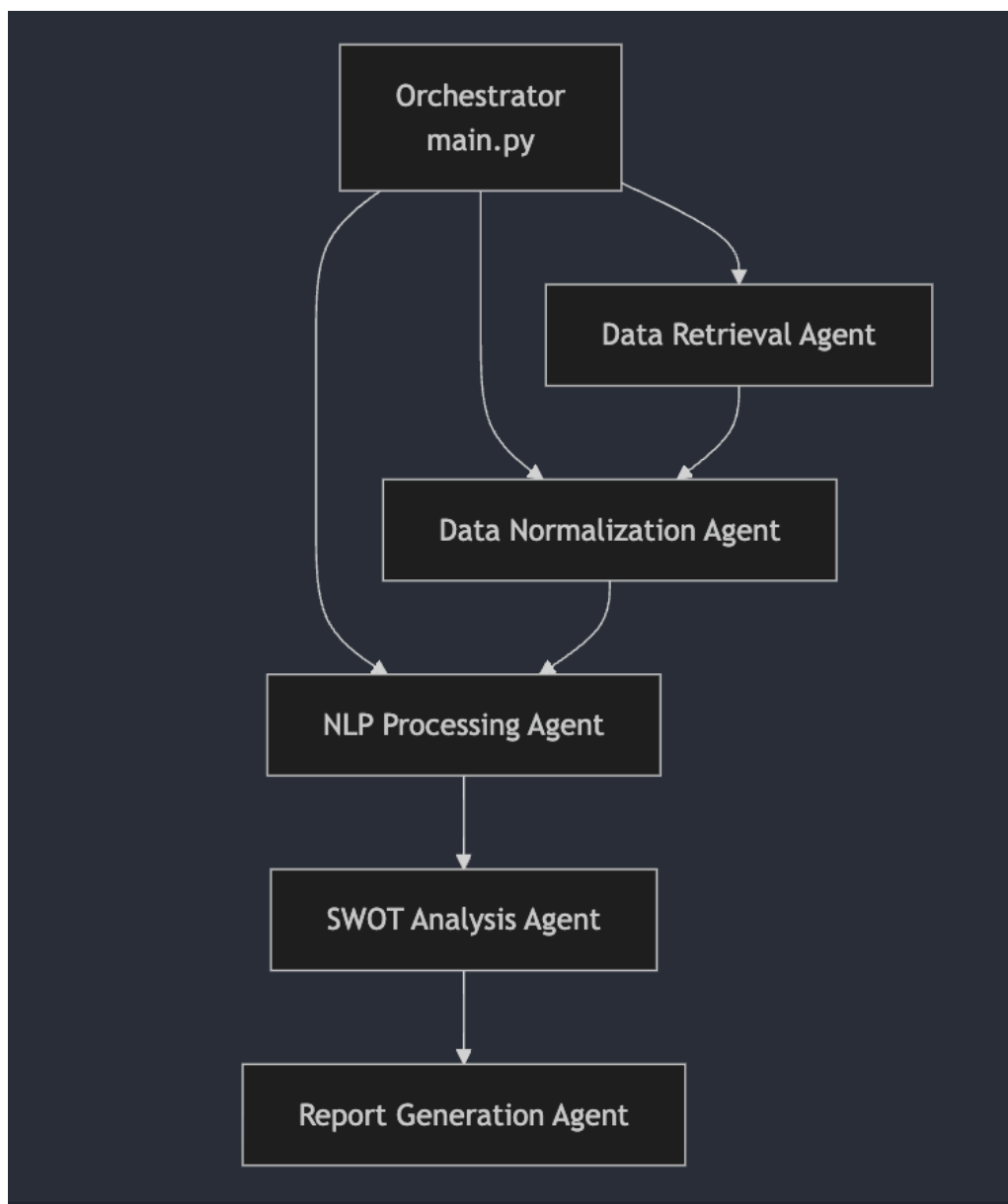
The system is built using a modular, multi-agent architecture. Each agent is responsible for a specific task in the data processing pipeline:

- Data Retrieval Agent:**
Queries external APIs (Crunchbase, LinkedIn, Google Custom Search, G2) to collect raw data about the target company.
- Data Normalization Agent:**
Converts the raw data from various sources into a consistent format, handling missing values and resolving conflicts.
- NLP Processing Agent:**
Enriches the normalized data by extracting summaries, named entities, sentiment, and other linguistic features.
- SWOT Analysis Agent:**
Aggregates the enriched profile and uses the OpenAI Chat Completion API (e.g., GPT-4 or GPT-3.5 Turbo) to generate a detailed SWOT analysis.

5. **Report Generation Agent:**
Compiles the final company profile and SWOT analysis into a comprehensive report.
 6. **Orchestrator (main.py):**
The central control module that coordinates the flow of data between agents.
-

3. Architectural Diagram

Below is an architectural diagram illustrating the interactions between the multi-agent components:



4. Justification for Chosen Technologies

4.1 LLM Selection

- **Chosen Model:** OpenAI's Chat Completion API (e.g., GPT-4 or GPT-3.5 Turbo)
- **Rationale:**
 - **Robust Language Capabilities:** The LLM is adept at synthesizing information and producing detailed, human-readable SWOT analyses.
 - **Flexibility:** Chat-based interactions allow for nuanced responses that are tailored to the detailed competitor profile.
 - **Ease-of-Integration:** The API-based approach minimizes infrastructure overhead while ensuring high-quality outputs.

4.2 Frameworks and Libraries

- **Programming Language:** Python (≥3.8)
 - **Justification:** Python's extensive ecosystem includes libraries for HTTP requests, NLP processing, and data manipulation.
- **HTTP & API Integration:**
 - **Libraries:** `requests` for API calls and `python-dotenv` for secure management of API keys.
- **Data Manipulation:**
 - **Library:** `pandas` for any tabular data operations if needed.
- **NLP Processing:**
 - **Libraries:**
 - `spaCy`: For Named Entity Recognition (NER) and text tokenization.
 - `TextBlob`: For sentiment analysis.
- **LLM Integration:**
 - **Library:** `openai` Python package for interfacing with the Chat Completion API.

4.3 Data Handling Strategies

Data handling in the Multi-Agent Competitor Analysis System is a multi-layered process that ensures the raw data collected from diverse external sources is accurate, consistent, and enriched before being passed to the Large Language Model (LLM) for SWOT analysis generation. The strategies include the following key components:

4.3.1 Multi-Source Data Extraction (ETL Approach)

- **Extraction:**

The system begins by querying multiple external APIs (e.g., Crunchbase, LinkedIn, Google Custom Search, G2) using the input company name. Each source returns data in its own format with varying levels of detail and structure.

- **Transformation:**

Since different data sources have unique schemas, the raw data is transformed into a common structure through the Data Normalization Agent. This step involves:

- **Standardizing Field Names:**

For example, while Crunchbase might return a field called "name" and "description", Google Custom Search may provide "title" and "snippet". The normalization process maps these into a unified schema.

- **Formatting and Cleaning:**

Textual data is cleaned by trimming extra spaces and removing unwanted characters. Non-string data is converted into appropriate formats (e.g., ensuring that features are always represented as lists).

- **Load:**

Once the data is transformed and normalized, it is loaded into a structured form (a set of dictionaries) that subsequent agents (e.g., NLP Processing and SWOT Analysis Agents) can easily consume.

4.3.2 Conflict Resolution and Missing Values

- **Conflict Resolution:**

When data is pulled from multiple sources, there may be overlaps or discrepancies in the information for a single company. The system employs a dedicated utility (in `utils/data_handler.py`) to resolve such conflicts:

- **Merging Data:**

If two sources provide different descriptions or names for the same company, the system may merge them. For example, if one source has a brief description and another a more detailed one, the conflict resolution logic will combine the information to preserve as much detail as possible.

- **Priority Rules:**

The system can be configured to prioritize certain sources (e.g., Crunchbase or LinkedIn) over others. This prioritization ensures that data from trusted sources is given greater weight in the final, merged profile.

- **Handling Missing Values:**

In cases where certain fields are missing or incomplete, the system uses default values or placeholder text. For example:

- If the company description is missing, the system might default to "No description available."

- For features that are expected to be a list but are missing or provided as a single string, the system ensures conversion into a proper list format.

- This systematic filling of missing values is essential to ensure that the downstream processes (e.g., NLP enrichment and SWOT analysis) receive a complete data set, thereby reducing the risk of errors or incomplete analyses.

4.3.3 Data Enrichment and Aggregation

- **NLP-Based Enrichment:**
Once the data has been normalized and conflicts resolved, the NLP Processing Agent further enriches the data:
 - **Summarization:**
The first sentence of a detailed description is extracted to form a summary.
 - **Entity Recognition and Sentiment Analysis:**
Tools like spaCy and TextBlob extract named entities and analyze the sentiment of the description, respectively. These additional data points provide richer context to the company profile.
- **Aggregation:**
After enrichment, the final company profile is aggregated into a comprehensive string that includes:
 - **Company Name**
 - **Detailed Description**
 - **Extracted Summary**
 - **Features (if available)**
 - **Additional Enriched Data (e.g., Named Entities and Sentiment)**
- This aggregation step ensures that when the profile is passed to the SWOT Analysis Agent (and ultimately the LLM), it contains all the relevant details required to generate a meaningful and accurate SWOT analysis.

4.3.4 Benefits and Trade-Offs

- **Benefits:**
 - **Consistency:**
Using a standardized schema for all incoming data helps maintain consistency across diverse sources.
 - **Accuracy and Completeness:**
Merging data from multiple sources and filling in missing values leads to a more comprehensive company profile.
 - **Modularity:**
By separating extraction, transformation, and loading into distinct steps handled by different agents, the system is more maintainable and scalable.

5. Discussion of Alternative Approaches and Trade-Offs

5.1 Monolithic vs. Multi-Agent Architecture

- **Monolithic Approach:**
 - **Pros:** Simpler to develop initially.
 - **Cons:** Difficult to maintain and scale as new data sources or features are added.
- **Multi-Agent Approach (Chosen):**
 - **Pros:**
 - Increased modularity and separation of concerns.

- Easier to extend and maintain individual agents without affecting the whole system.
 - Better scalability.
- **Cons:**
 - Increased complexity in orchestrating communication between agents.

5.2 API-Based LLM vs. Self-Hosted LLM

- **API-Based LLM (Chosen):**
 - **Pros:**
 - Reduced overhead and maintenance.
 - Access to state-of-the-art models (e.g., GPT-4) with minimal setup.
 - **Cons:**
 - Dependency on external service and associated costs.
- **Self-Hosted LLM:**
 - **Pros:**
 - Potential cost savings and control over the model.
 - **Cons:**
 - Requires significant computational resources.
 - Involves complex setup and maintenance.